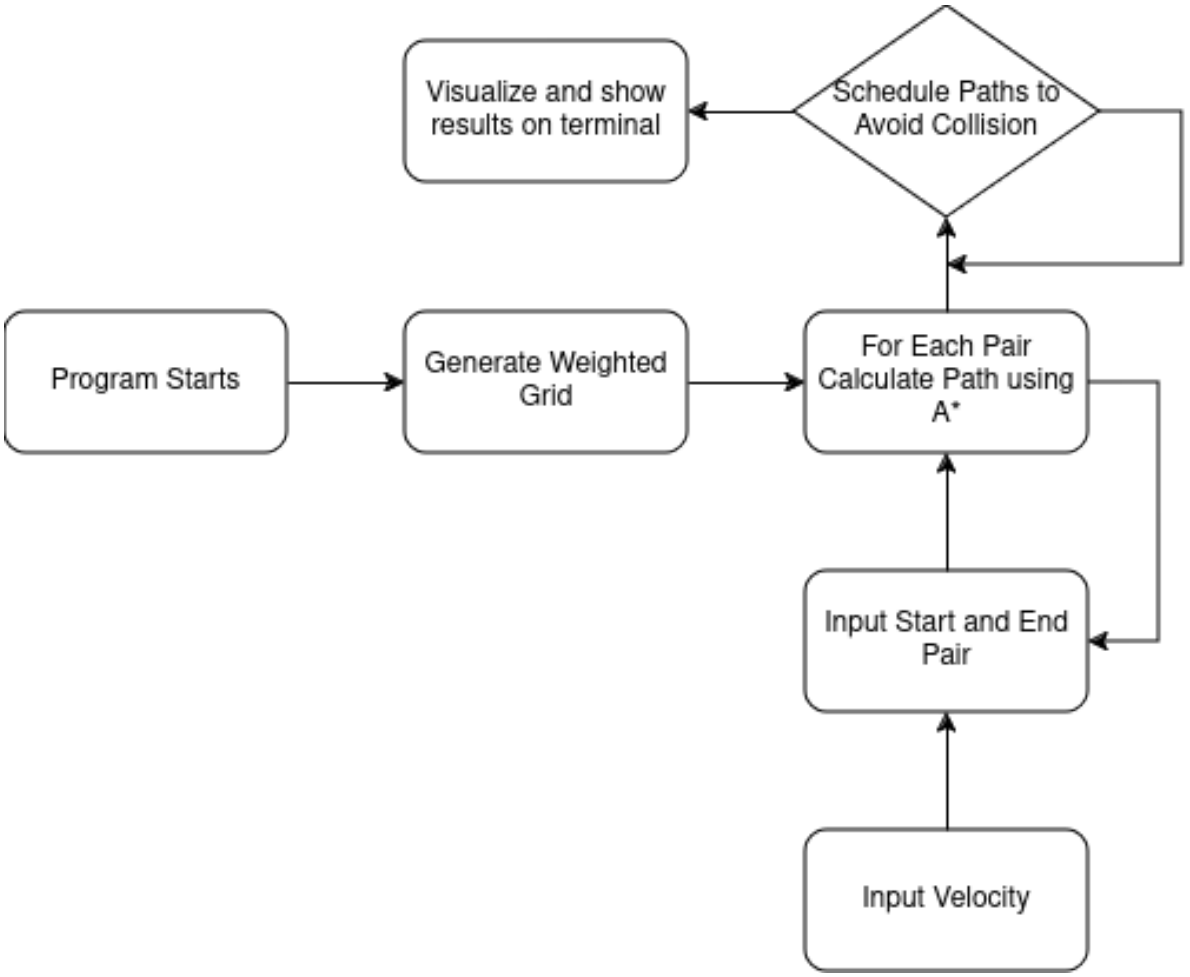
TASK 1: Finding shortest Path in 3D



This Python script is a complex yet elegant solution for simulating pathfinding and scheduling in a weighted 3D grid environment. The primary aim of the program is to find optimal paths between given start and end points in a grid where certain regions carry varying traversal costs. It also schedules these paths to avoid temporal conflicts, ensuring that multiple agents can traverse the grid without colliding in time or space. By incorporating advanced algorithms, user interaction, and 3D visualization, the program offers a comprehensive and user-friendly simulation framework.

The process begins with generating a weighted grid using the `generate_weighted_grid` function. This grid is represented as a three-dimensional NumPy array, where each cell has a cost associated with traversing it. While most cells remain unweighted and free, a randomly selected 10% of the grid points are assigned weights between 1 and 10. These weights represent traversal costs and create a more challenging environment for the pathfinding algorithm. This grid serves as the foundation of the simulation, introducing complexity and variability to the paths that need to be calculated.

The core of the pathfinding process lies in the implementation of the A* algorithm within the `astar` function. A* is a widely-used heuristic-based algorithm that efficiently calculates the shortest path between two points. In this implementation, the algorithm uses the Euclidean

distance as a heuristic to guide the search towards the destination. It evaluates potential paths by considering both the actual cost of traversing a path so far and an estimated cost to the target. At each step, it prioritizes the path with the lowest combined cost, ensuring an optimal route. The algorithm also handles grid boundaries and dynamically updates the scores for neighboring nodes when a better path is found. If no valid path exists due to obstructions or high-cost regions, the algorithm gracefully terminates, returning a failure for that specific pair of points.

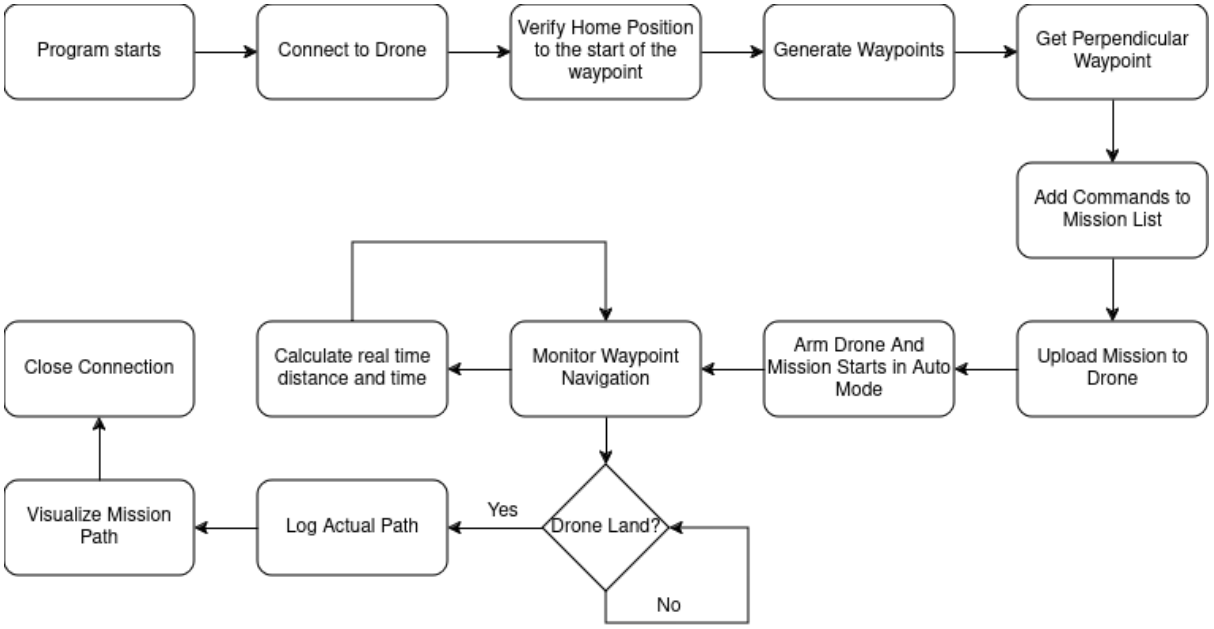
Once the paths are computed, they are processed through the `schedule_paths` function to ensure that they can coexist without temporal conflicts. This scheduling accounts for the possibility that two paths might cross or overlap in space, leading to potential collisions. To resolve this, a temporal map is maintained to track when and where each grid cell is occupied. If a conflict is detected, the affected path is delayed to avoid overlap. This process ensures that each agent's traversal is smooth and collision-free, even in a shared environment. The function outputs both the adjusted paths, annotated with their respective times, and the total travel times for each path.

The final component of the script focuses on visualization, using the `plot_paths_with_weights` function. This function creates a 3D representation of the paths and their interaction with the weighted grid. Each path is visualized in a unique color, while the weights along the path are depicted as scaled and color-mapped points. This visualization not only highlights the routes taken by the agents but also emphasizes the regions of the grid with higher traversal costs. By saving the generated plot as an image, the program ensures that the results can be reviewed or shared for further analysis.

The user interacts with the program through a series of prompts in the `main` function. The user specifies the velocity of travel and the number of start-end point pairs to simulate. For each pair, the program calculates a path, schedules it, and integrates it into the final visualization. If a path cannot be found for a particular pair, the program provides feedback to the user, indicating the limitations posed by the grid configuration. Once all paths are processed, the total travel times for each are displayed, offering a quantitative measure of the traversal efficiency.

The script's design ensures that each component works seamlessly with the others, resulting in a robust and efficient simulation. By integrating pathfinding, scheduling, and visualization into a cohesive workflow, the program offers a powerful tool for exploring navigation in complex environments. Its modular structure also makes it adaptable to future enhancements, such as dynamic grid updates, real-time path recalculations, or even integration with physical robotics. Through its thoughtful implementation and rich feature set, this script serves as both a practical simulation tool and an educational example of advanced computational techniques.

Task 2: Mission Planner through waypoints



This Python script demonstrates a drone mission planning and execution process using the DroneKit and MAVLink libraries. The program integrates waypoint navigation, perpendicular waypoint generation, real-time monitoring, and visualization of the drone's flight path. It leverages computational geometry to calculate bearings and distances for waypoint adjustments and offers insights into the drone's status, such as remaining distance and estimated mission time.

The process begins with establishing a connection to the drone's autopilot system via DroneKit. The connection string specifies the communication channel (e.g., a local simulation environment or a physical drone). Upon successful connection, the program verifies the home position of the drone to ensure accurate relative positioning for the mission.

Waypoints are generated relative to the drone's home position using the `get_location_offset_meters` function. This function calculates new geographical coordinates by offsetting a given location by specific distances in the north and east directions. These waypoints are distributed along a zigzag pattern for this mission, with alternating eastward offsets.

To add complexity to the mission, a perpendicular waypoint is introduced at a specified location. Using the `calculate_bearing` function, the bearing between two sequential waypoints is computed. A perpendicular offset is then determined, and a new waypoint is inserted using the `get_perpendicular_location` function. This waypoint creates an additional detour in the mission path, showcasing the flexibility of the waypoint management system.

Once all waypoints are defined, they are added to the mission command list. The final waypoint is designated as a landing command, instructing the drone to terminate the mission with a descent. The commands are uploaded to the autopilot for execution.

The mission begins by arming the drone and initiating the first waypoint navigation. During the mission, the program continuously monitors the drone’s progress. The distance to the next waypoint and the total remaining distance are calculated in real-time using the `calculate_distance` function, which employs the haversine formula for geographical distance measurement. The estimated time to mission completion is derived from the drone's current ground speed and the remaining distance.

While the drone follows the mission path, its actual trajectory is logged for comparison with the planned route. This data is visualized in the `plot_mission_path` function, which plots the planned and actual paths on a 2D graph. The plot highlights any deviations, offering valuable insights for mission analysis and refinement.

The mission concludes with the drone landing at the designated endpoint. Once the drone is disarmed, the program outputs the mission's completion details, including the total distance traveled and the estimated time. Finally, the connection to the drone is closed gracefully.